

1. THE PROBLEM — OEM TOOL DEPENDENCY

- Oil Pump & Air Pump ECUs shipped with a proprietary Chinese OEM tool (candebug.exe)
- Source code unavailable — closed binary
- Uses USBCAN hardware — not our PEAK adapters
- No automation possible — GUI-only operation
- Protocol was completely undocumented
- Any firmware update required OEM presence
- Could not integrate into our existing workflow
- Zero control over error handling or logging
- Single point of failure for production flashing

2. REVERSE ENGINEERING APPROACH — CAN LOG CAPTURE & ANALYSIS

STEP 1 — Passive CAN Capture

- Connected PEAK PCAN-USB to the vehicle bus
- Ran OEM tool to flash a known-good ECU
- Captured full CAN bus log (timestamped frames)

STEP 2 — Frame Identification

- Isolated frames on extended IDs 0x180005FF and 0x1800FF05 (tool ↔ ECU pair)
- Confirmed broadcast ID 0x1800FFFF on connect
- Identified 0x1800EEEE as NAK / error channel

STEP 3 — Pattern Analysis

- All frames exactly 8 bytes long
- Byte[7] = XOR of bytes [0..6] → checksum
- Byte[0] cycles: 0x04 → 0x01 → 0x02 → 0x01... then 0x00, then 0x04 → ... for each block

STEP 4 — Confirm ACK Mechanism

- ECU replies with the EXACT same 8 bytes on its own CAN ID (echo-ACK model)
- Timeout = no echo within window → retry/fail

3. PROTOCOL DECODED — 8-BYTE CUSTOM CAN FRAME

Frame layout: [TYPE][P1][P2][P3][P4][P5][P6][XOR8]

TYPE = 0x00 → Heartbeat (wakeup) or End-of-Block (with block_id in P2)
TYPE = 0x01 → Data frame (toggle A) payload = P1..P6 (6 bytes firmware)
TYPE = 0x02 → Data frame (toggle B) payload = P1..P6 (6 bytes firmware)
TYPE = 0x03 → Close session payload = CRC bytes
TYPE = 0x04 → Block init / erase payload = addr(4B) + size(2B)
XOR8 = P0 XOR P1 XOR P2 XOR P3 XOR P4 XOR P5 XOR P6 (bytes 0..6)

Broadcast wakeup (live ECU):

FF 02 <ecu_addr> 00 00 00 00 <XOR>
Sent on 0x1800FFFF — ECU ACKs with FF 82 ...
ECU reboots to bootloader in ~300 ms

Heartbeat byte formula:

$hb = (0x3F4000 - base_addr) \div 0x2000 + 1$
Auto-computed from Intel HEX base address

Erase + data sequence per block:

04 <addr 4B> <size 2B> → erase ACK
01 <6 bytes> → data toggle A
02 <6 bytes> → data toggle B
01/02 ... (repeat) → all firmware bytes
00 00 <block_id> ... → end-of-block ACK
04 <CRC addr> → write CRC
01 <CRC verify> → verify + close

4. KEY DISCOVERY — VER1 STRIDE ANOMALY & OEM BLOCK PARSER

VER1 firmware flashing failed with our first tool:

- Tool sent 5 sequential 16 KB blocks
- ECU booted but CRC mismatch on reboot

Root cause found by comparing HEX files:

- VER1 Intel HEX has non-contiguous records
- Gap between records > half the record size → called a 'stride anomaly'
- OEM tool splits at anomaly boundaries producing 7 blocks (not 5)

Detection algorithm implemented:

if $(addr - prev_addr) > prev_byte_count \div 2$:
→ stride anomaly → new block boundary

Result — VER1 produces 7 OEM-compatible blocks:

Block IDs: 6,5,5,4,3,2,1 (ID formula driven)
Block ID = $(0x3F4000 - addr) \div 0x2000$
Last block always ID = 1

VER2/VER3: no anomalies → 5 sequential blocks ✓

5. INTEGRATION INTO JSW FLASHXPRT & FULL HARDWARE CONTROL